

WOLF: Weight-Level OutLier and Fault Integration for Reliable LLM Deployment

Chong Wang , Wanyi Fu , Jiangwei Zhang , *Member, IEEE*, Shiyao Li , Rui Hou ,
Jian Yang , *Senior Member, IEEE*, and Yu Wang , *Fellow, IEEE*

Abstract—The rapid advancement of Transformer-based large language models (LLMs) is presenting significant challenges for their deployment, primarily due to their enormous parameter sizes and intermediate results, which create a bottleneck in memory capacity for effective inference. Compared to traditional DRAM, Non-Volatile Memory (NVM) technologies such as Resistive Random-Access Memory (RRAM) and Phase-Change Memory (PCM) offer higher integration density, making them promising alternatives. However, before NVM can be widely adopted, its reliability issues, particularly manufacturing defects and endurance faults, must be addressed. In response to the limited memory capacity and reliability challenges of deploying LLMs in NVM, we introduce a novel low-overhead weight-level map, named WOLF. WOLF not only integrates the addresses of faulty weights to support efficient fault tolerance but also includes the addresses of outlier weights in LLMs. This allows for tensor-wise segmented quantization of both outliers and regular weights, enabling lower-bitwidth quantization. The WOLF framework uses a Bloom Filter-based map to efficiently manage outliers and faults. By employing shared hashes for outliers and faults and specific hashes for faults, WOLF significantly reduces the area overhead. Building on WOLF, we propose a novel fault tolerance method that resolves the observed issue of clustering critical incorrect outliers and fully leverages the inherent resilience of LLMs to improve fault tolerance capabilities. As a result, WOLF achieves segment-wise INT4 quantization with enhanced accuracy. Moreover, WOLF can adeptly handle Bit Error Rates as high as 1×10^{-2} without compromising accuracy, in stark contrast to the state-of-the-art approach where accuracy declines by more than 20%.

Index Terms—Large language models (LLMs), quantization, fault tolerance, and non-volatile memory (NVM).

Received 30 June 2024; revised 30 April 2025; accepted 26 June 2025. Date of publication 17 July 2025; date of current version 10 September 2025. This work was supported in part by the National Natural Science Foundation of China Distinguished Young Scholars Program under Grant 62125208; and in part by the National Natural Science Foundation of China under Grant 62325405, Grant U19B2019, Grant U21B2031, Grant 92364201, Grant 62222102, and Grant 62171023. Recommended for acceptance by Y. Hu. (Chong Wang and Wanyi Fu contributed equally to this work.) (Corresponding author: Jiangwei Zhang.)

Chong Wang, Wanyi Fu, Shiyao Li, Jian Yang, and Yu Wang are with the Department of Electronic Engineering, Tsinghua University, Beijing, China.

Jiangwei Zhang and Rui Hou are with the State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China (e-mail: jwzhang.pitt@gmail.com).

Digital Object Identifier 10.1109/TC.2025.3587957

I. INTRODUCTION

THE emergence of Large Language Models (LLMs) has revolutionized artificial intelligence, showcasing remarkable generalization capabilities and substantial commercial potential [1], [2]. Despite their advantages, deploying LLMs poses significant challenges due to their vast parameter sizes and the substantial intermediate results they generate, which strain memory capacity during inference [3]. To address these challenges, Non-Volatile Memory (NVM) technologies such as Resistive Random-Access Memory (RRAM) and Phase-Change Memory (PCM) have emerged as promising alternatives to traditional DRAM, offering higher integration density. However, widespread adoption of NVM is hindered by reliability issues, including manufacturing defects and endurance faults, which must be resolved to ensure consistent performance [4], [5], [6].

An effective method to correct hard faults in NVM is to integrate the addresses of faulty bits into a map and store the corresponding data in additional bits [7]. The state-of-the-art error correction map, FLOWER [7], is a bit-level map based on Bloom filters. However, **we observe that, for LLM storage, a weight-level map is more suitable, allowing for lower overhead integration of faulty weight addresses and corresponding weight-level fault tolerance methods designed for LLMs.** Additionally, we have identified the following challenges when current fault tolerance methods are applied to LLM deployment on NVM:

- 1) **Lack of Differentiation:** Outliers are weights with absolute values significantly higher than the average, typically accounting for about 0.1% to 1% under Mean Squared Error (MSE) optimization [8], [9]. Current methods often do not distinguish between faults in outliers and regular weights, which is crucial for LLMs. Faults in outliers generally have a greater impact on LLM inference accuracy compared to regular weights due to their significant deviation and higher influence on predictions [8], [9].
- 2) **Local Clustering of Outliers:** Outliers tend to cluster locally, which can lead to critical faults. Existing mechanisms often distribute error correction capacity evenly, making them less effective at handling clusters of outlier faults that exceed the allocated correction capacity for specific memory regions.
- 3) **Inherent Fault Tolerance of LLMs:** Many LLMs have inherent tolerance to non-critical weight faults, which traditional memory-focused fault tolerance methods, such

as FLOWER [7], often overlook. These methods focus on absolute error correction without considering the inherent tolerance of LLMs.

- 4) **Challenges with Dynamic Faults:** Some fault tolerance approaches for DNNs, such as WESCO [10], leverage this inherent resilience but may struggle with dynamic faults, like endurance faults, and their effectiveness still needs improvement.

Besides integrating faulty weights, during low-bit quantization of LLMs, there is a high quantization error issue caused by outliers. By saving addresses of outliers, different quantization methods can be applied to them compared to regular weights, potentially reducing the overall quantization error of LLMs. **We observe that the addresses of outliers can be integrated in a manner similar to that of faulty weights. Moreover, we also observe that with Bloom filters technology, addresses of different types of data can share hash functions, thereby reducing the area overhead required to store these integrated addresses or maps.**

To address these challenges, we propose WOLF (an acronym for ‘Weight-Level OutLier and Fault Integration’), a novel approach that effectively manages outliers and faults. WOLF employs shared hashes to integrate outliers and faults and utilizes a specified hash to filter faults from the combined set of outliers and faults. This method reduces WOLF’s area overhead by nearly 40% compared to maps that separately integrate outliers and faults. This use of shared hash in Bloom filter technology can also be applied to other types of data, provided the proportion of such data is relatively low, typically below 1%. For example, unstructured ultra-sparse sparsity patterns in DNNs can also be integrated.

As depicted in Figure 1, WOLF demonstrates distinct advantages in area overhead compared to existing methods like ECP [11], ArchShield [12], and FLOWER [7] across various Bit Error Rates (BER). At a BER of 10^{-5} , WOLF (FT) shows the smallest area overhead, outperforming ArchShield, ECP, and FLOWER. As BER increases, FLOWER begins to outperform ArchShield, but WOLF (FT) continues to maintain significantly lower overheads, achieving reductions of 69%-92% compared to FLOWER. In the range of 10^{-5} to 10^{-3} , WOLF (OT) maintains a minimal overhead of 0.78%. Even at a BER of 10^{-2} , where outliers become more dominant, WOLF (OT) increases to only 3.1%. Overall, considering both fault tolerance (FT) and outlier (OT) overheads, WOLF (FT+OT) achieves a total overhead of just 1.27% to 4.78%, highlighting its efficiency and effectiveness.

WOLF enables segment-wise 4-bit quantization or mixed quantization for outliers and regular weights at the tensor level rather than the channel or group-of-channels level employed in Ref. [13], [14]. This method outperforms the latest tensor-level advancement [9], by $\sim 1\%$ in accuracy and offers enhanced compatibility with existing GPUs and accelerators.

WOLF’s innovative approach to fault tolerance not only prioritizes the correction of incorrect outliers but also employs a nuanced method for handling hard faults across regular weights. This comprehensive fault management strategy ensures that WOLF maintains model accuracy even at high BER levels of

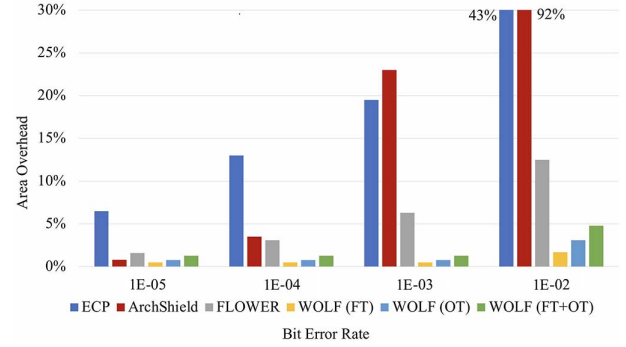


Figure 1. The area overhead for different methods to identify faults in a typical 4GB memory across various Bit Error Rates (BER). For WOLF, FT stands for fault tolerance, OT for outliers, and FT+OT for combined overhead.

up to 1×10^{-2} , significantly outperforming existing methods that may experience accuracy declines of more than 20% under similar conditions. In conclusion, we present the following key contributions:

- 1) We introduce WOLF, a novel weight-level Bloom Filter-based map that simultaneously records outliers and faults.
- 2) We develop a WOLF-based tensor-wise quantization method that allows for distinct strategies for outliers and regular weights, supporting both segment-wise INT4 and mixed INT4/INT8 modes.
- 3) We introduce a WOLF-based fault tolerance method that effectively handles hard faults, with a focus on managing incorrect outliers.
- 4) We design a WOLF architecture that supports the deployment of LLMs, detecting and integrating outliers and faults. It facilitates updates to WOLF and provides an end-to-end flow for memory access.
- 5) We provide comprehensive evaluations of our proposed quantization and fault tolerance techniques, demonstrating their superior performance compared to state-of-the-art quantization and fault-tolerance mechanisms.

II. BACKGROUND

A. LLMs

LLMs are at the forefront of advancements in Natural Language Processing (NLP), with models like OpenAI’s GPT-4 [15], Google’s BERT [16], and Facebook’s LLAMA [17] achieving impressive levels of text comprehension, generation, and translation. The newly released GPT-4 Turbo by OpenAI [15] enhances these capabilities further, offering greater efficiency and a substantial 128K context window for sophisticated AI tasks. Additionally, models such as GPT-4o [15] and various multimodal models [17] are pushing the boundaries of AI capabilities even further. LLMs operate in two primary phases: the resource-intensive training phase and the subsequent inference phase, where the pre-trained model is fine-tuned for specific applications. Low-bit quantization techniques significantly reduce the computational and memory requirements of LLMs [9], yet they introduce substantial challenges

in terms of quantization errors, particularly with outliers. Moreover, the enormous parameter sizes of these models exacerbate storage and memory access latency issues during deployment, making efficient memory utilization and fault tolerance critical for reliable performance.

In contrast to CNNs, where both weights and activation values are key, LLMs prioritize weight quantization during inference due to their extensive use of fully connected layers and less demanding activation computations [1]. These activations are typically calculated once and reused. LLMs also exhibit greater deviation than CNNs, making maintaining accuracy with low-bit quantization like INT4 more challenging, especially due to outliers. Poor outlier management can lead to significant quantization errors. Furthermore, when weights are deployed to unreliable NVMs, faults can further degrade accuracy, particularly faults within outliers.

B. Non-Volatile Memories

NVM technologies such as RRAM and PCM offer significant advantages in integration density and power consumption. These two-terminal devices achieve a cell size as small as $4F^2$, denser than DRAM's $6F^2$, and support 3D integration for even higher densities.

However, NVMs face critical reliability challenges: manufacturing faults and limited endurance. Reliability depends on maintaining distinguishable resistance states (resistance windows) between different states. Over time, switching cycles narrow these windows, leading to endurance faults. Process variations (PV) cause inconsistencies in optimal RESET currents, applied uniformly across devices, leading to varied endurance. This PV results in some “weak” cells having significantly lower endurance than the average.

When NVM stores weights, activation values, and KV cache values in LLMs, hard faults can degrade inference accuracy. Activation values occupy a smaller proportion, and KV cache values are dependent on the input length. Hard faults manifest as stuck-at-right (SAR) and stuck-at-wrong (SAW) faults. Traditional fault tolerance methods correct some faults but are less effective for LLMs. Additionally, LLMs require critical memory resources, making high extra area overheads unacceptable. Additionally, given the large memory requirements of LLMs, even a small proportional increase can lead to significant costs.

III. FLOWER+FAME

Figure 2 illustrates an example of FLOWER [7], a bit-level fault map based on Bloom filters [18], designed to integrate memory fault addresses with low overhead. In this example, the memory has 2^5 rows, each with 6 bits. Using two hash functions, the rows are mapped to 2^3 vectors, each also 6 bits long. According to the principles of Bloom filters, Hash1 and Hash2 map Row 1 and Row 10 to Fault Vector 1 (FV1). In FV1, Bit 1 and Bit 4 are set to 1, while the rest remain 0. Similarly, Hash2 and Hash1 map Row 1 and Row 20 to FV2, and Hash1 and Hash2 map Row 10 and Row 20 to FV3. However, during interpretation, Bit 1 in Row 10 is mistakenly identified as a fault which is a false positive.

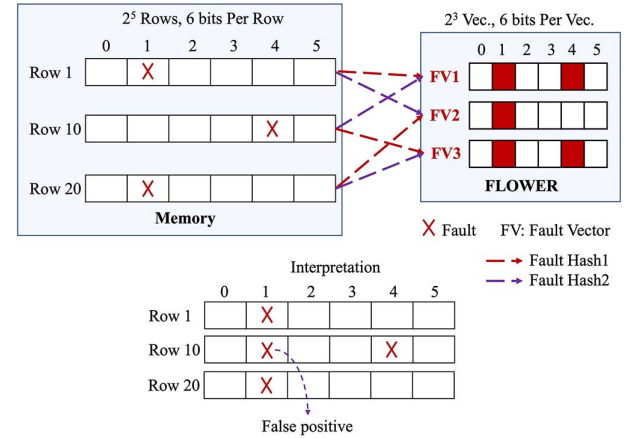


Figure 2. An example of FLOWER [7]. Hash1 and Hash2 map Row 1 and Row 10 to Fault Vector 1 (FV1), setting Bit 1 and Bit 4 to 1. Bit 1 in Row 10 is mistakenly identified as a fault, illustrating a false positive.

To reduce false positives caused by hash collisions in FLOWER, FLOWER employs the Minimum Cumulative Intersection (MinCI) [7], an algorithm that generates hash functions with fewer collisions. MinCI creates bit masks that map row addresses into hashes. For example, the address 10001111 can be hashed to 1000, preserving the first four bits. The bit masks generated by MinCI have minimal cumulative intersection with each other, resulting in fewer hash collisions.

FaME [7] builds on FLOWER by providing a low-overhead error correction method. For each bit-level fault and false positive in a row, FaME assigns a single bit as its replica. Given that the proportion of false positives is generally low, this approach ensures that, excluding FLOWER's area overhead, each fault on average requires only 1 bit.

For LLM applications, the mapping and fault tolerance methods must handle weights as the primary unit of correction. This focus allows for significant compression of FLOWER's area, reducing the overhead associated with fault management in LLMs. LLMs exhibit unique characteristics such as the presence of outliers and a higher impact of faults within these outliers on inference accuracy. Additionally, LLMs inherently possess a degree of fault tolerance for non-critical weights. By optimizing the fault map specifically for LLMs, taking into account these characteristics, we can achieve efficient fault tolerance with minimal overhead.

IV. THE DESIGN OF WOLF MAP

A. Rule of Shared Hash

WOLF is a specialized mapping scheme designed to handle both outliers and faults in NVM. By employing a *shared-hash* mechanism that combines outliers and faults within the same submaps, WOLF effectively reduces the number of submaps by nearly half compared to maintaining separate submaps for each category—thereby significantly lowering overall overhead. To distinguish faults from outliers, a small set of dedicated fault hashes is introduced, which creates a separate fault map and

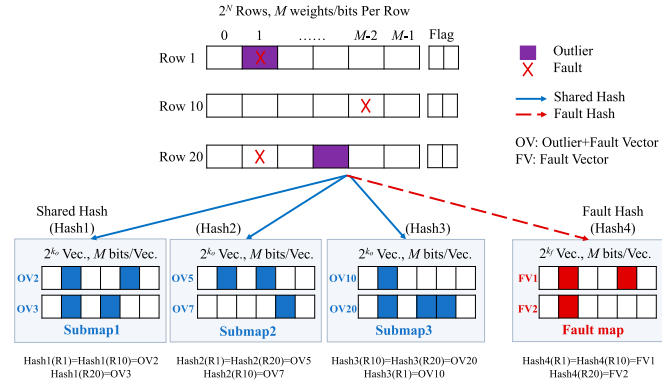


Figure 3. An example of WOLF. The WOLF map uses three hash functions to map outliers and faults from the memory to its Submaps and one hash function to map faults to its Fault Map. ‘OV’ and ‘FV’ represent Outlier+Fault Vector and Fault Vector, respectively. Each row in the memory corresponds to an OV in a Submap, and each weight corresponds to a bit in the vector. If a weight is an outlier or has a fault, the corresponding bit in the vector is set to ‘1’, marked in blue. The Fault Map marks positions with faults as ‘1’, marked in red.

isolates faulty weights with minimal storage cost. In the following subsections, we explain how the quantization filters manage outliers and faults together, while the fault filters specifically identify and isolate faults within this unified set.

B. Quantization Filter

1) *WOLF Organization*: WOLF comprises two components: the overall part and the fault part. As shown in Figure 3, outliers and faulty values in the memory are denoted by purple squares and ‘X’ marks, respectively. The memory is mapped to three Submaps through three distinct shared hash functions, forming the overall part. Each memory row corresponds to an Outlier+Fault Vector (OV) in a Submap, with each weight corresponding to a bit in the vector. If a weight is an outlier or contains a fault, the corresponding bit in the vector is set to ‘1’. A separate hash function maps the memory to a Fault Map, representing the fault part, which we will discuss in the following subsection.

In Figure 3, the hash functions below the Submaps and Fault Map illustrate the relationship between the row addresses in the memory and the vector addresses in the map. For instance, Hash1 maps Row 1 and Row 10 of the memory to OV2 in the Submap. The 1st weight in Row 1 is an outlier (with fault), and the $(M-2)$ th weight in Row 10 is faulty. Therefore, the 1st and $(M-2)$ th positions in OV2 are marked as ‘1’ (shown in blue), while the other positions are ‘0’ (shown in white).

Assuming the memory uses N bits for its row address, there are 2^N rows, each containing M weights. Each weight is represented by an n -bit integer, making the memory capacity $2^N \times M \times n$. For the overall part, the memory is mapped to arrays with k_o -bit row addresses, resulting in array sizes of $2^{k_o} \times M$. k_o is much smaller than N , and the Bloom filters employ hash functions to map each of the 2^N memory rows to the corresponding rows within the 2^{k_o} rows of the Bloom filter storage. While this records outliers and faults in the submaps,

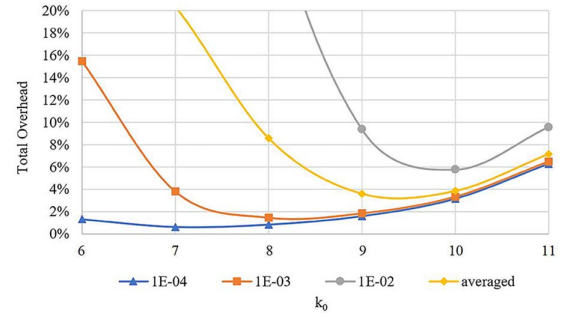


Figure 4. Overhead across varying k_o under different BERs.

it introduces some false positives due to multiple rows being mapped to the same location.

To reduce false positives, we employ techniques from Ref. [7], including the organization of subarrays and MinCI method, ensuring a uniform hash function distribution across memory addresses. WOLF utilizes n_1 sub-arrays, each with a distinct hash function. For example, Row r in the memory could be mapped to OV_j in Submap i ($i = 1, \dots, n_1$), where j ($j = 0, \dots, (2^{k_o} - 1)$) represents the address of the corresponding vector in Submap i of Row r , i.e., $OV_j = \text{Hash}_i(R_r)$. The position of an outlier or fault in the original row can be obtained by performing a bitwise AND operation on the corresponding vectors in the n_1 sub-arrays.

In order to determine k_o and n_1 , we must consider both the area overhead of the sub-arrays and the false positives. A larger k_o increases the number of rows in each sub-array but reduces hash collisions, while a larger n_1 reduces false positives but increases area overhead. For example, when quantizing the OPT-6.7B model [2], if regular weights and outliers are quantized in INT4 with different scales, we can plot the total overhead that avoids any accuracy drop against the values of k_o in Figure 4 and n_1 in Figure 5, respectively. The results show that larger k_o and n_1 perform better at higher BERs, while smaller k_o and n_1 are more effective at lower BERs. Note that in Figure 5, as the number of maps increases, the false positive rate drops rapidly. However, when the number of maps becomes large, the reduction in false positives no longer compensates for the overhead caused by the increased number of maps, leading to a rise in total overhead. At a value of 1×10^{-4} , the false positive rate for a single map is already very low, so as the number of maps increases, the total overhead also increases. Considering the full lifetime of a memory where the BER changes from 1×10^{-4} to 1×10^{-2} , we take the average overhead over the lifetime. We choose $k_o = 9$ and $n_1 = 4$, which results in the minimum total average overhead.

2) *WOLF Interpretation*: As illustrated in Figure 6, WOLF interprets the positions of outliers and faulty weights. When accessing a specific row in memory (e.g., Row 20), WOLF employs multiple shared hash functions to locate the corresponding OVs in the submaps. As shown in Figure 6(a), these submaps, along with the fault map, are stored in separate memory subarrays, allowing for concurrent access. A bitwise AND

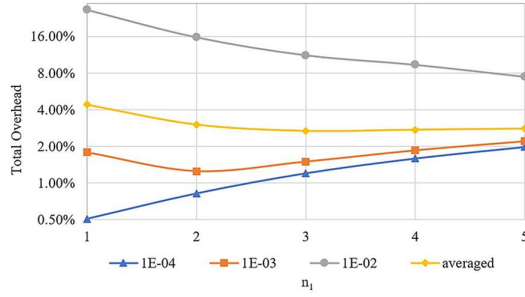


Figure 5. Overhead (log-scale) across varying n_1 .

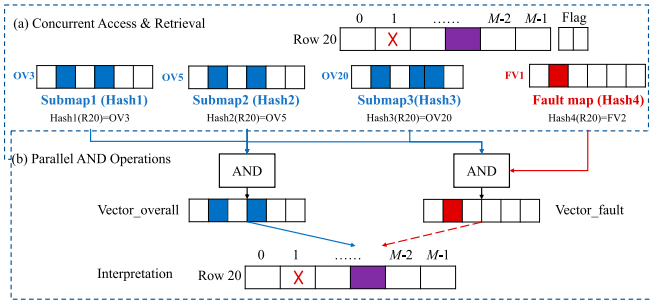


Figure 6. An example of WOLF's decoding process for the positions of outliers and faulty weights. The process involves (a) concurrently accessing Row 20 in memory and locating the corresponding OVs and FV using hash functions, and (b) performing parallel AND operations to calculate Vector_overall and Vector_fault.

operation on the OVs produces a vector_overall, which indicates the positions of outliers and faulty weights, without distinguishing between the two. In the subsequent subsection, we will provide a simple and efficient method used to differentiate between outliers and faults. As shown in Figure 6(b), this design enables the parallel execution of AND operations across the OVs and FV directly at the memory controller, significantly reducing the time complexity associated with these operations.

Outliers, determined by pre-set thresholds, are decoded using a unique quantization method. In contrast, faults undergo corrective measures. This streamlined process ensures robust memory access in LLM accelerators, preparing for detailed elaboration in Section VII-C.

As BER increases, hash collisions may increase in submaps and fault maps, which could result in regular weights being quantized with outliers' scales. To demonstrate the effectiveness of our shared hash technique, we plot the false positive rate of quantization and fault tolerance caused by faults' hash collisions while fault rate increases, and compare it to FLOWER with the same overhead in Figure 7. The results show that the false positive rate caused by faults' hash collisions and hash sharing between faults and outliers are both significantly lower than that of FLOWER with the same overhead.

C. Fault Filter

In Section VI-A, we found that even minimal faults significantly impact the accuracy of quantized LLMs, highlighting the need for precise fault identification during quantization.

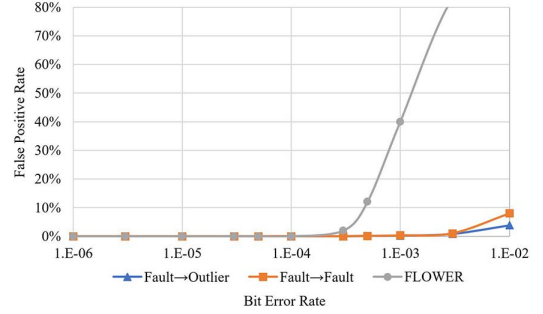


Figure 7. False positive rates of WOLF-based quantization, fault tolerance and FLOWER with the same overhead.

This subsection introduces an innovative fault tolerance mechanism anchored in our unique quantization filter. Notably, faults are less frequent than outliers, often by orders of magnitude. This allows us to efficiently incorporate fault data into the WOLF map with minimal overhead during outlier quantization. Our method uniquely identifies fault locations through the quantization filter, requiring only a few additional fault filters to distinguish faults from the combined pool of outliers and faults.

In our method, we incorporate n_2 additional fault filters. When n_2 is set to 1, using the method in Section VI allows us to perform 1×10^{-2} error correction with less than 1% overhead, which is lower than the overhead of using an alternative fault map. Therefore, we set n_2 to 1. We use a rule where a weight is deemed an outlier or faulty if all n_1 hashes for quantization at a position are '1'. If not, it's considered a regular value without fault. To identify faults more precisely, we check if all $n_1 + n_2$ hashes are '1'. If so, the weight is classified as faulty; otherwise, it's seen as correct. This approach efficiently locates and corrects faults while preserving the quantization process's effectiveness.

For instance, in Figure 3, the Fault Map marks the three weights in the memory that have faults with the corresponding positions set to '1' (shown in red). In Figure 6, to interpret the location of the faulty weights, an AND operation between OVs and FV produces vector_fault, which indicates the positions of faulty weights. The vector_overall and vector_fault allow us to interpret outliers and faults in a memory row. WOLF tags outliers and faulty weights, using fault filters to distinguish faults. However, it cannot determine if these faults are related to outliers. We propose a simple encoding method to identify incorrect outliers.

To manage the inaccuracies in faulty weights, we leverage the signed INT4 encoding scheme, where both 0000 and 1000 represent zero. We reserve 0000 to indicate zero and repurpose 1000 to denote faulty non-outliers. For incorrect outliers, we don't use the 1000 encoding. This approach allows us to distinguish effectively between faulty non-outliers and incorrect outliers. While this method sacrifices the value of faulty non-outliers, we will validate that the impact on LLM inference accuracy is negligible. This validation, along with the methods to correct faulty weights, is discussed in Section VI-B. This encoding strategy ensures that we can accurately identify and

differentiate faulty non-outliers and incorrect outliers, enabling precise fault management and error correction.

In general, for the fault part, the memory is mapped to a fault array with a k_f bit row address, resulting in an array size of $2^{k_f} \times M$. k_f is much smaller than N . The Bloom filter uses the hash function to map each of the 2^N memory rows to the 2^{k_f} rows of the Bloom filter storage. As the n_1 hashes for quantization are shared with fault tolerance, the false positive rate for faults is significantly reduced.

V. WOLF-BASED QUANTIZATION

In this section, we first mathematically analyze the impact of segment-wise quantization on overall quantization error when applied to outliers. Then, we introduce the WOLF-based quantization method, including both segment-wise and mixed quantization methods.

A. Quantization Errors

For uniform quantization, assuming that the weight is in the interval $[V_{\min}, V_{\max}]$, n -bit integer quantization is used. The quantization step size is

$$\Delta_U = \frac{V_{\max} - V_{\min}}{2^n} \quad (1)$$

The quantization error, e_U , for uniform quantization ranges between $-\Delta_U/2$ and $\Delta_U/2$.

Let us assume that it is possible to distinguish outliers and represent outliers and ordinary values by mixed precision or different data types. By setting a certain proportion of large values as outliers (typically 0.1-1% for LLMs), the minimum value of the remaining common values is still $V_{\min}$, and the maximum value is reduced to $OV_{\max}$. The updated quantization step size for ordinary values is

$$\Delta_{OV} = \frac{OV_{\max} - V_{\min}}{2^n} \quad (2)$$

Similarly, the quantization error, e_{OV} , for uniform quantization for ordinary values ranges between $-\Delta_{OV}/2$ and $\Delta_{OV}/2$. Since the uniformly distributed quantization error is proportional to the quantization step size, the quantization error for ordinary values is reduced by

$$T_{OV} = \frac{V_{\max} - V_{\min}}{OV_{\max} - V_{\min}} \quad (3)$$

times. If uniform quantization is also used for outliers, the quantization step size of outliers is

$$\Delta_{OT} = \frac{V_{\max} - OT_{\min}}{2^n} \quad (4)$$

where $OT_{\min}$ is the minimum value of the outliers. If we set $OT_{\min} = OV_{\max}$, the quantization error for outliers is reduced by

$$T_{OT} = \frac{V_{\max} - V_{\min}}{V_{\max} - OV_{\max}} \quad (5)$$

times. If we use Mean Square Error (MSE) to measure the impact of quantization

$$MSE = \int (x - \hat{x})^2 p(x) dx \quad (6)$$

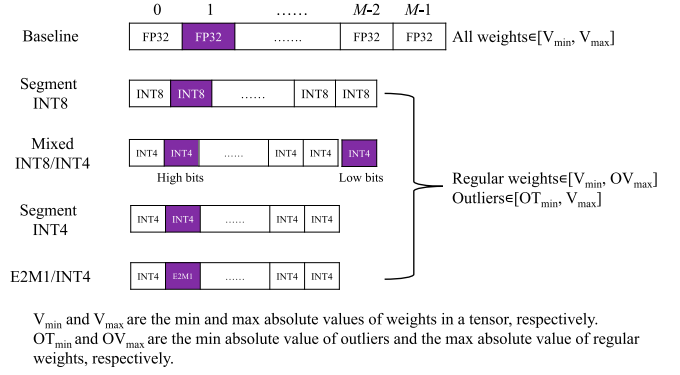


Figure 8. WOLF-based quantization methods.

where x is the weight and $\hat{x}$ is an approximation of x . $p(x)$ is the probability density function.

Non-uniform quantization methods can reduce quantization error by altering the data format. For example, outliers are processed using a mix of float and int, whereas regular weights are quantized in INT4 format [9], [19].

B. The Proposed Quantization Methods

WOLF-based Quantization methods build upon the WOLF map. As shown in Figure 8, these methods are below:

- 1) *Segment-wise INT8 Quantization*: Both outliers and regular weights are quantized using 8-bit integers.
- 2) *Mixed INT8/INT4 Quantization*: Outliers utilize 8-bit integers, while regular weights are quantized with 4-bit integers. The high bits of outliers are in INT4, with the low bits positioned at the end of the row. Both are computed simultaneously, after which the high bits undergo a shift and are added to the low bits.
- 3) *Segment-wise INT4 Quantization*: Every weight, be it an outlier or regular, is quantized using 4-bit integers.
- 4) *E2M1/INT4 Quantization*: Employs the E2M1 format for outliers, combining a sign bit, two exponent bits, and a mantissa bit to extend the numerical range, ensuring precise outlier management.

We prefer Segment-wise INT4 Quantization due to its minimal bitwidth, compatibility with GPU architectures, and computational efficiency. To effectively handle the broad range of outliers in LLMs with Segment-wise INT4 Quantization, we introduce a scaling factor, α , which adjusts the quantization range of outliers relative to regular weights, ensuring a balanced and efficient quantization approach. Additionally, there is an overall scaling factor, β .

Segment-wise INT4 Quantization benefits from its low bit-width, supporting parallel computation of all weights without the need for sequential calculations or alignment required in mixed quantization methods where INT8 weights must be added after aligning the high and low bits. For outliers, after computing the decimal value, it is multiplied by the product of α and β ; for regular weights, the decimal value is multiplied by β .

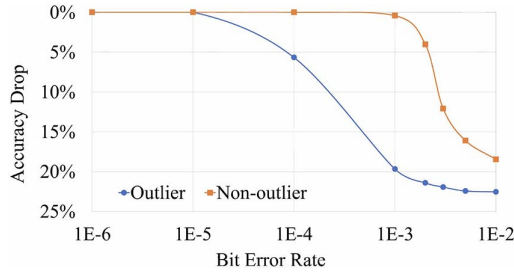


Figure 9. Impact on accuracy when faults are concentrated in outliers versus non-outliers.

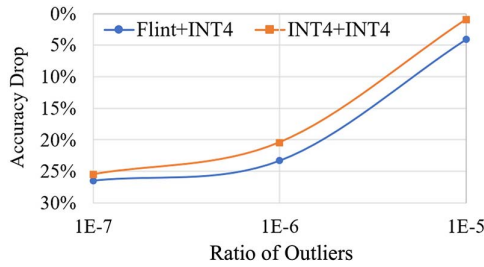


Figure 10. Shift in the accuracy of the LLM when a certain number of faults are concentrated in the larger outliers.

VI. WOLF-BASED FAULT TOLERANCE

This section first experimentally verifies the impact of different types of faults on LLM inference accuracy and studies the distribution of outliers. We then propose different fault tolerance methods for outliers and non-outliers, which are highly cost-effective and efficient.

A. Impact of Hard Faults

Our experimental setup evaluates the impact of faults on LLMs, focusing on BERs from 10^{-6} to 10^{-2} . We tested the OPT-6.7B model on the PIQA dataset, identifying 0.5% of weights as outliers. We examined scenarios where faults impact only outliers (*Outlier*) or only regular weights (*Non-outlier*), isolating *Outlier* faults to those weights. Results, as depicted in Figure 9, show a significant accuracy drop for *Outlier* at a BER of 10^{-4} , much lower than for *Non-outlier* at 2×10^{-3} , highlighting the critical vulnerability of outliers to faults.

Moreover, we evaluated the impact of larger outliers on the accuracy of the model, as shown in Figure 10. The BER is merely 10^{-7} , targeting top outliers in the range of 10^{-7} to 10^{-5} in the OPT-6.7B model. Consequently, the number of incorrect outliers is confined to a few hundred. Quantization of these outliers in both INT4 and Flint formats led to a 25% accuracy reduction, highlighting the critical impact of larger outliers with faults. Notably, the combination of Flint and INT4 led to a more pronounced decrease in accuracy compared to segment-wise INT4, attributable to the elevated quantization error inherent in the Flint format.

To prove the effectiveness of WOLF's error correction method, we observed a local clustering of outliers in multiple LLMs. For OPT-6.7B, with 512 4-bit weights per row and an

outlier ratio of 0.5%, and randomly distributed BER of 10^{-3} and 10^{-2} , we can calculate that on average, each row has 0.01 and 0.1 incorrect outliers, respectively. However, based on our experiments, approximately 0.2% and 3.3% of the rows, which we refer to as “weak rows,” contain 2 or more incorrect outliers. These weak rows require additional protection; otherwise, the remaining incorrect outliers can easily degrade the LLM's accuracy.

B. Enhancing Fault Tolerance in LLMs

Due to the significant difference in the impact of faults in outliers and non-outliers on LLM accuracy, we address these two types of faults separately.

Outliers in memory rows of LLMs often cluster, with some rows exhibiting a higher-than-average count, as illustrated in Section VI-A. This can lead to an excess of incorrect outliers, challenging the error correction capacity. To address this challenge, we propose a highly cost-effective method. Considering that the majority of faulty outliers have only one hard fault, which could be either stuck-at-right (SAR) or stuck-at-wrong (SAW), we allocate a flag bit for each faulty outlier. If the faulty outlier is SAR, the flag bit is set to 0; if it is SAW, the flag bit is set to 1. In the rare cases where a faulty outlier has two or more faults, if all faults are either SARs or SAWs, the previous method can correct them all. If the faults include both SAR and SAW, we prioritize correcting the higher-order bit faults to minimize the impact. For example, with a BER of 10^{-3} and an outlier proportion of 0.5%, the probability of an INT4 weight having more than two faults that include both SAR and SAW is approximately 7.5×10^{-11} . By prioritizing the correction of high-order bit faults, the impact of such cases is negligible.

For faulty non-outliers, as explained in Section IV-C, we encode faulty non-outliers as 1000 to distinguish them from incorrect outliers. Based on the experimental results in Figure 9, we know that LLMs can inherently tolerate faulty non-outliers up to $\text{BER}=10^{-3}$. Given that weight values in LLMs typically adhere to a normal distribution centered around zero, assuming unknown weights are near zero is a relatively safe assumption [20]. Therefore, we sacrifice these faulty weights and integrate the addresses of faulty non-outliers into the WOLF map. Since the encoding of faulty non-outliers (1000) needs protection, we assign a flag bit to each faulty non-outlier. If there are only SARs, the flag bit is set to 1. For example, with a BER of 10^{-3} and an outlier proportion of 0.5%, the probability of a faulty non-outlier (INT4) having more than two faults that include both SAR and SAW is as low as 3.0×10^{-6} . When interpretation identifies the addresses of faulty non-outliers within a row, the corresponding weights are reset during decoding.

VII. MEMORY ARCHITECTURE

A. WOLF Architecture

The architecture of WOLF is designed to handle the complexities of LLMs effectively. Quantized weights are written into multiple banks using either Non-bit slice or bit slice techniques. The WOLF map, which integrates outliers and faults, is

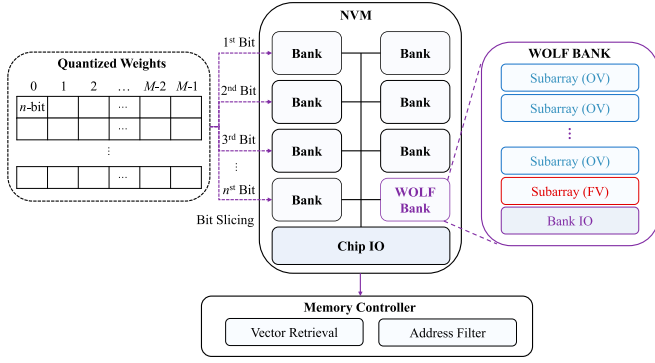


Figure 11. The architecture design of WOLF.

stored separately in a dedicated bank. The memory controller plays a crucial role in this architecture, supporting both vector retrieval and address filtering.

Non-Bit Slice Approach: A traditional solution is the non-bit slice approach. In this method, OVs and FV are stored in different rows, mapped by hash functions. The WOLF architecture sequentially activates these rows and reads the vectors into a temporary buffer in the memory controller. Once all vectors are loaded into the buffer, the memory controller performs the required logic operations to obtain the interpretation results.

Bit Slice Approach: As depicted in Figure 11, another approach is the bit slice approach. Quantized weights are bit-sliced and distributed across different subarrays. The WOLF architecture uses techniques similar to FLOWER [7] to open multiple rows across subarrays simultaneously. Adjusting the sensing threshold allows for performing AND operations directly in-memory. Results are obtained with minimal comparisons due to efficient in-memory logic operations.

Comparison of Latency and Power: The non-bit slice approach requires sequentially activating rows and transferring data to the memory controller buffer, resulting in much higher latency due to multiple row activations and data transfers. The bit slice approach, on the other hand, activates multiple rows simultaneously and performs in-memory logic operations, achieving lower latency due to parallel processing and efficient in-memory computation, including synchronization overhead. Moreover, the non-bit slice approach consumes more power due to increased activity from sequential data transfers and multiple row activations. In contrast, the bit slice approach has lower power consumption due to reduced data transfers and efficient in-memory operations, benefiting from NVM's lower operational power. Thus, while the non-bit slice approach is simpler to implement, the bit slice approach is more suitable for WOLF due to its substantial improvements in latency and power efficiency.

B. Deployment and Updates

This subsection outlines the deployment of LLMs and the initial construction of the WOLF map, focusing on detecting hard faults and identifying outliers. Additionally, it explains how to detect hard faults and integrate faults at runtime.

For NVM, manufacturing faults are detected using techniques like the sneak-path [21] and verification read [22] techniques. Fault addresses are temporarily recorded and not immediately updated to the WOLF map. When the quantized model is written into memory, both these outliers and faults are mapped into the WOLF map. As outlined in Section IV-C, WOLF employs a specific encoding method for faulty weights. Faulty non-outliers are encoded as 1000 to distinguish them from incorrect outliers. Incorrect outliers avoid using the 1000 encoding. This ensures that the positions of both outliers and faults are accurately marked. Each faulty weight is also allocated an auxiliary bit to correct hard faults. This encoding effectively captures the exact locations of incorrect outliers, enabling precise fault management and error correction. This deployment phase, including both quantization and recording, is a one-time process for LLMs, performed statically and can be completed offline.

At runtime, new hard faults are detected using the verification read [22] technique after each write or periodically. The faults are integrated into WOLF, updating both OVs and FV. If the fault occurs at a non-outlier, it is encoded as 1000. Since endurance faults typically occur in the mid to late stages of the memory's lifetime, the frequency of faults is very low in the early stages. Therefore, the frequency of periodic verification reads can be low initially and gradually increase over time. Additionally, because NVM writes inherently require verification reads to ensure correct data writing, there is no need for extra verification reads to detect faults during these operations.

C. Implementation Details

This subsection elaborates on WOLF's implementation, highlighting the memory controller's key components as depicted in Figure 11. It includes an Address Filter (AF), responsible for managing rows with outliers or faults.

In Algorithm 1, we show a pseudo code of WOLF's implementation. The AF requires only 1 bit per row. Under INT4 and INT8 quantization, the area overhead is $\frac{1}{4M}$. For example, when $M = 128$, the overhead for AF is merely 0.2% and 0.1%, respectively. Our proposed fault tolerance method allocates 1 flag bit per faulty weight, whereas FLOWER assigns 1 bit per faulty bit. This means that for INT4 or INT8 quantization, WOLF's overhead is significantly lower. Specifically, not considering the integration of outliers in quantization, WOLF's fault tolerance overhead is about 25% and 12.5% of FLOWER's overhead for INT4 and INT8, respectively.

Faults occurring in the map can be corrected using ECC. WOLF's weight-level integration occupies only about 25% and 12.5% of the space required by FLOWER [7], leading to proportionally smaller ECC overhead. For (64,72) ECC, this adds about 12.5% additional overhead to WOLF, resulting in an absolute value of about 25% and 12.5% of FLOWER's ECC overhead. If we consider shared hashes to further reduce WOLF's area overhead, WOLF's advantage becomes even more significant, as shown in Figure 1.

Algorithm 1: WOLF: Pseudocode

```

1: Parameters:
   • Number of weights in each row of the memory  $M$ 
   • Number of bits in the submap row address  $k_0$ 
   •  $n_1 + n_2$  sub-arrays with shape of  $2^{k_0} \times M$ :
      $\text{map}_{n_1+n_2}[2^{k_0}, M]$ 
2: Inputs:
   • Row address  $r$ 
3: Functions:
   •  $\text{filter}(x)$ : Address filter ( $x$  is a row address)
   •  $\text{hash}_i(x)$ : Hash functions ( $x$  is a row address)
   •  $\text{outlier}(x)$ : Outlier weight decoder ( $x$  is quantized weight)
4:  $\text{result} \leftarrow$  Quantized Data in Row  $r$ 
5:  $\text{flags} \leftarrow$  Flag Data in Row  $r$ 
6: if  $\text{filter}(r) = \text{true}$  then
7:   Hashed Addresses:  $h_i = \text{hash}_i(r)$  for  $i = 1, 2, \dots, n_1 + n_2$ 
8:   Outlier Mask (size  $M$ ):  $\text{OM} = \prod_{i=1}^{n_1} \text{map}_i[h_i]$ 
9:   Fault Mask (size  $M$ ):  $\text{FM} = \prod_{i=1}^{n_1+n_2} \text{map}_i[h_i]$ 
10:   $k \leftarrow 0$ 
11:  for all  $j$  from 1 to  $M$  do
12:    if  $\text{FM}[j] = \text{true}$  then
13:      if  $\text{result}[j] \neq 1000$  then
14:        // Xor every bit of  $\text{result}[j]$  with the corresponding flag
15:         $k \leftarrow k + 1$ 
16:         $t \leftarrow \text{xor}(\text{result}[j], 1111 * \text{flags}[k])$ 
17:         $\text{result}[j] \leftarrow \text{outlier}(t)$ 
18:      else
19:         $\text{result}[j] \leftarrow 0$ 
20:      end if
21:    else
22:      if  $\text{OM}[j] = \text{true}$  then
23:         $\text{result}[j] \leftarrow \text{outlier}(\text{result}[j])$ 
24:      end if
25:    end if
26:  end for
27: end if
28: return  $\text{result}$ 

```

VIII. EVALUATION

To evaluate our WOLF-based quantization and fault tolerance methods, we have developed a simulator¹ and implemented it in Pytorch [23]. We have tested this on several models, including GPT2-XL [24], BLOOM-7B1 [25], OPT-6.7B [2], LLAMA3-8B [17] and LLAMA2-13B [26], using the PIQA [27] and ARC [28] datasets with PyTorch on RTX 3090 GPU. For all the models mentioned above, we have utilized state-of-the-art (SOTA) checkpoints from the Hugging Face repositories [29].

We first conducted a validation of the quantization capabilities offered by WOLF. For the quantization aspect, we benchmarked our method against INT4 quantization, ANT [19], and SOTA techniques like Olive [9]. We also presented results for

segment-wise INT4 quantization, mixed INT8/INT4 quantization, and segment-wise INT8 quantization. Our simulation framework accurately models the behavior of LLMs under a variety of conditions. Outliers are the weights whose magnitudes exceed a data-driven threshold, comprising about 0.5% of all weights. A sensitivity sweep confirms that this ratio is near-optimal for accuracy. The LLMs undergo quantization according to a predefined scheme. In contrast to our method (E2M1), which applies segment-wise uniform quantization for both outliers and regular weights, other methods utilize a uniform range between their respective maximum and minimum values.

Regarding fault tolerance, we compared our method with leading-edge solutions such as WESCO [10] and FLOWER+FaME, hereinafter referred to as FaME [7]. Additionally, we detailed the performance, area, and power overheads associated with WOLF's quantization and fault tolerance mechanisms. The fault model for NVM will be provided in Section VIII-B.

A. WOLF-Based Quantization

As shown in Table I, using FP32 results as our baseline, we emphasize segment-wise INT4 quantization due to its significant reduction in storage and computation demands for LLMs. For this quantization, our approach offers two outlier quantization strategies: Ours(INT4) and Ours(E2M1), with the latter inspired by Olive [9]. We also showcase results for Ours(Mixed) representing INT8/INT4 mixed quantization and Ours(INT8) for segment-wise INT8. We measure model accuracy using Accuracy norm, which adjusts option scores based on their length to account for biases introduced by varying option lengths. Including the overhead of submaps, Ours(INT4), Ours(E2M1), Ours(MIXED) and Ours(INT8) have the average bit width of 4.03, 4.03, 4.08 and 8.03. Note that the overhead bit width of Ours(MIXED) is calculated based on the line containing most faulty outliers.

In Table I, the quantization accuracy of the ANT method for the three LLMs is not ideal, with a significant decrease in accuracy compared to the baseline. This is due to ANT's uniform quantization method and range for both outliers and ordinary values, which results in higher quantization errors. Olive's quantization performance is excellent, especially with the GPT2-XL model, but it experiences a degradation with the BLOOM-7B1, OPT-6.7B and LLAMA series models. For the GPT2-XL model, all quantization methods of 'Ours' achieved baseline performance. However, for BLOOM-7B1, OPT-6.7B, LLAMA3-8B, and LLAMA2-13B, there is a gap of about 3 percent between our segment-wise INT4 quantization and the baseline. Ours(INT4) generally yields results that are better than those obtained with the E2M1 format. This superiority is especially pronounced in the case of the OPT-6.7B model, where our approach demonstrates a distinct improvement. With the exception of the GPT2-XL model, Ours(INT4) surpasses Olive by approximately one percent. Furthermore, it does not necessitate any additional hardware support, providing it with a substantial edge. While Olive compromises a few outliers

¹Codes are available for reference at <https://github.com/g4sd/wolf>

TABLE I
COMPARISON OF OUR QUANTIZATION METHOD WITH THE BASELINE (FP32) AND OTHER EXISTING METHODS

LLM	GPT2-XL		BLOOM-7B1		OPT-6.7B		LLAMA3-8B		LLAMA2-13B	
Dataset	PIQA	ARC	PIQA	ARC	PIQA	ARC	PIQA	ARC	PIQA	ARC
FP32	70.62	28.58	77.75	43.60	76.50	34.64	80.79	53.41	80.52	49.06
INT4	67.08	26.11	69.15	28.50	59.79	24.40	51.63	26.79	49.40	28.24
ANT	62.19	24.91	60.88	27.22	54.30	21.08	51.52	26.28	49.46	28.07
Olive	70.02	28.50	73.72	37.80	72.91	30.97	78.89	48.72	79.11	45.90
Ours(INT4)	70.18	28.41	76.12	40.61	73.83	31.57	79.22	50.43	79.38	47.10
Ours(E2M1)	69.97	28.41	73.23	35.92	69.21	28.75	78.51	49.23	79.11	45.31
Ours(Mixed)	69.91	28.41	76.82	41.13	76.22	33.36	79.60	50.60	79.60	48.81
Ours(INT8)	70.62	28.58	77.75	43.77	76.61	34.73	80.52	53.07	80.41	49.91

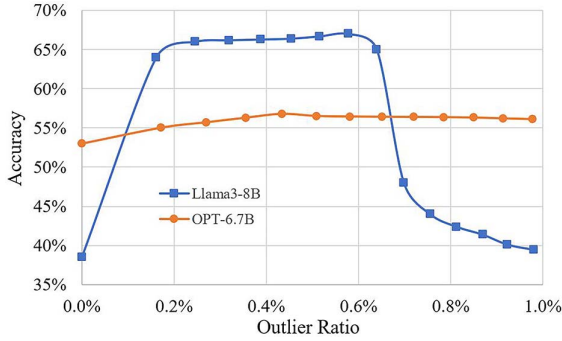


Figure 12. Average accuracy of LLAMA3-8B and OPT-6.7B across different outlier ratios.

in outlier-outlier pairs and victims in outlier-victim pairs, our approach, backed by the WOLF map, sacrifices significantly fewer standard weights and retains all outliers.

The results of Ours(INT8) are identical to the baseline, while those of Ours(Mixed) are slightly superior, albeit with alignment issues. In scenarios where resources are abundant and accuracy is critical, Ours(INT8) is an excellent choice. However, for resource-constrained scenarios, Ours(INT4) strikes a good balance between overhead and accuracy.

Our WOLF-based quantization stands out for its performance across the transformer-based models. It effectively mitigates the majority of accuracy degradation observed with other methods, providing a robust and efficient solution for weight quantization.

We performed a sensitivity study to determine the optimal outlier ratio. Each model was evaluated with ratios sweeping from 0% to 1%, and we selected the setting that maximized the average accuracy over all test datasets. As shown in Figure 12, the best ratio for LLaMA-3 8B is 0.42%, whereas OPT-6.7B attains its peak accuracy at 0.58%. Both optima lie close to 0.5%, which we therefore adopt as the default ratio in all subsequent experiments.

B. Fault Model

For NVM technologies, such as PCM and RRAM, the fault model reflects their unique physical characteristics and operational conditions. Unlike DRAM, where certain fault patterns are more prevalent due to architectural and layout considerations, NVM faults encompass a broader spectrum. These include manufacturing defects and endurance faults, leading to

hard faults. While DRAM faults may cluster due to specific architectural vulnerabilities, NVM faults are generally considered to be more uniformly distributed, although localized wear-out or access hotspots can still lead to non-uniform fault distributions in practice. Our fault model for NVM, as detailed in Ref. [30], device faults caused by these factors can be classified into hard and soft faults based on the type of fault, and into static and dynamic faults based on the time of occurrence. If there are severe ‘weak’ cells that are likely to result in soft faults, they can be treated as hard faults for detection purposes. By using our scheme in conjunction with ECC, we address both hard and soft faults.

Hard faults includes both manufacturing and endurance-related faults. Manufacturing faults, primarily hard faults, are modeled as being uniformly distributed throughout the memory cells, with the overall error rate defined by the BER. For endurance-related faults, we posit that the lifespan of each memory cell follows a normal distribution characterized by a mean number of writes and a standard deviation. The coefficient of variation (CoV) is employed to account for process variation, reflecting the inherent variability in cell durability and the potential for non-uniform fault distributions due to localized wear-out or concentrated access patterns.

We examine a range of BER from 10^{-6} to 10^{-2} . This range is selected to ensure WOLF’s robustness across both current and future memory technologies [7], [10].

C. WOLF-Based Fault Tolerance

Figure 13 demonstrates the OPT-6.7B model’s performance on NVM, following the fault model detailed in Section VIII-B. We compare five scenarios: no correction (None), (72, 64) Hamming ECC (abbreviated as ECC) [31], WESCO [10], FaME [7], and our WOLF method. In these scenarios, the area overhead for ECC is 12.5%, WESCO allocates several bits per row with an area overhead ranging from 1.5% to 4.0%, FLOWER’s area overhead ranges from 1.6% to 12.5%, and WOLF’s fault tolerance area overhead is between 0.5% and 1.5%. The total overhead for WOLF, covering both quantization and fault tolerance, is between 1.3% and 4.8%.

Without any correction mechanisms, model accuracy begins to decline by 2% at a BER of 7×10^{-5} , with a significant drop to 21% coverage as BER increases from 3×10^{-4} to 10^{-2} . With ECC protection, the model maintains flawless accuracy up to a BER of 3×10^{-4} . Both WESCO and FaME effectively sustain model accuracy up to a BER of 10^{-4} . Beyond this point, up to a BER of 10^{-2} , WESCO’s accuracy drops by

TABLE II
EVALUATION OF WOLF'S HARDWARE OVERHEAD VARYING TYPES AND MANUFACTURING PROCESSES

Memory Types	Read	Overhead (INT4)	Overhead (INT8)	Cache Miss Latency
32nm 8MB ReRAM	5.7 ns / 0.01 nJ	3 ns / 0.1 nJ	2 ns / 0.05 nJ	2 ns
90nm 512Mb diode-selected PCRAM	60 ns / 0.03 nJ	40 ns / 0.2 nJ	27 ns / 0.1 nJ	2 ns
0.18μm 4Mb MOSFET-selected ReRAM	8 ns / 0.04 nJ	4 ns / 0.2 nJ	3 ns / 0.1 nJ	2 ns

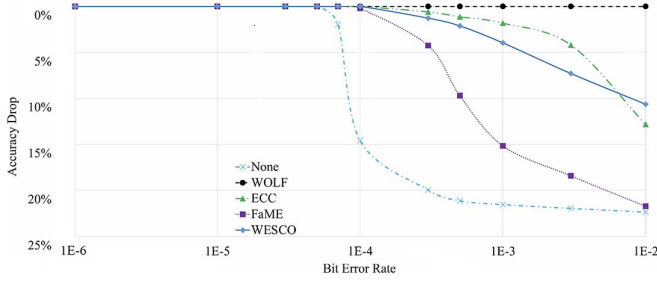


Figure 13. Comparison of accuracy between WOLF-Based Fault Tolerance and other methods at various BERs on OPT-6.7B.

approximately 11%, while FaME experiences a more severe decline of 22%. In stark contrast, WOLF showcases exceptional resilience, preserving accuracy without any loss even at the highest tested BER of 10^{-2} . Given WOLF's higher fault tolerance capability and significantly lower area overhead compared to other methods, it demonstrates its superior fault tolerance capabilities in NVM environments.

D. Overhead

We compare the overheads of WOLF-based quantization and fault tolerance in terms of power and performance to comparable methods. In Figure 1, we have compared the area overheads of ECP [11], ArchShield [12], FLOWER [7], and WOLF. WOLF-based fault tolerance shows significant reductions compared to FLOWER, achieving reductions of 69% to 92%. Even when including the overhead for outlier integration, WOLF remains more efficient, with overall area overheads being 20% to 80% lower than FLOWER. This demonstrates WOLF's superior ability to integrate fault tolerance and quantization strategies while minimizing space impact.

We evaluate our methods on a 32nm 8MB ReRAM, a 90nm 512Mb diode-selected PCRAM, and a 0.18μm 4Mb MOSFET-selected ReRAM with NVSim [32], [33]. A cache miss in the AF incurs latency due to reading from the AF. A cache hit in the AF requires additional operations, such as vector retrieval from the WOLF map and in-memory logic computation, which introduces extra latency. The additional latency and energy introduced by WOLF are listed in Table II.

Moreover, in Figures 14 and 15, we compared the latency and power consumption of WOLF and FLOWER working on 32nm 8MB ReRAM with the task of inferring OPT-6.7B [2] quantized in INT4 and INT8. It's shown that WOLF can reduce 7% of read latency and 52% read energy compared to FLOWER in INT4, while reducing 4% of latency and 38% energy compared to FLOWER in INT8.

Updates to the WOLF map are required only upon detection of a new hard fault. Given the infrequency of faults, a 4GB

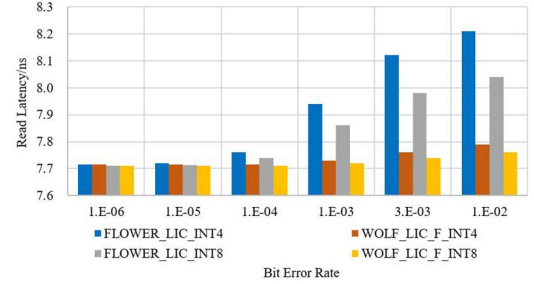


Figure 14. Comparison of read latency during inferring INT4/INT8 quantized OPT-6.7B across varying BER.

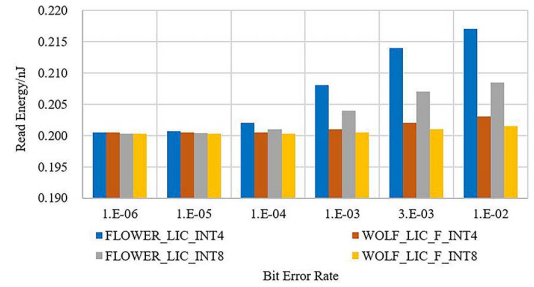


Figure 15. Comparison of read energy during inferring INT4/INT8 quantized OPT-6.7B across varying BER.

memory module would typically experience a few new faults on average each hour. The associated bandwidth, power, and performance overheads are minimal and can be considered negligible.

Figure 16 shows the performance overhead for FLOWER and WOLF across BERs from 10^{-6} to 10^{-2} . Labels 'LIC', 'QF', and 'F' represent logic-in-memory, quantization with fault tolerance, and fault tolerance alone. 'WOLF_LIC_QF' indicates WOLF with logic-in-memory and quantization for fault tolerance. 'WOLF_F', comparable to FLOWER, maintains a performance overhead under 3%. Logic-in-memory adds a minor 0.3% benefit, while FLOWER may see up to a 20% performance drop due to longer vector lengths and higher fault probabilities per vector. At a 10^{-6} BER, the main overhead is from AF latency. 'WOLF_LIC_QF' and 'WOLF_QF' incur 4%-5% overhead, with outliers being $\sim 0.5\%$ of weights, leading to frequent WOLF map accesses. This highlights the low overhead of the proposed methods.

IX. RELATED WORK

A. Quantization

Quantization aims to reduce weight precision in DNNs to enhance performance and reduce complexity. Techniques range

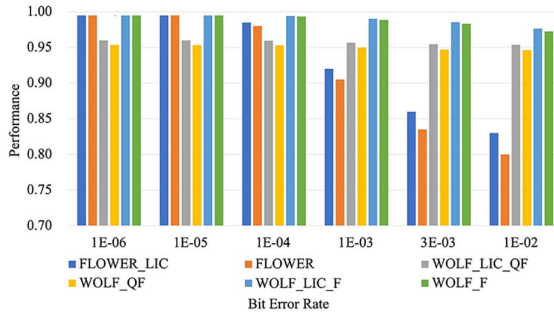


Figure 16. Comparison of performance overhead between FLOWER and WOLF across varying BER on OPT-6.7B.

from uniform and logarithmic to non-uniform quantization, each with its computational efficiency [34]. While traditional networks use FP32, quantization, including quantization-aware training (QAT) and post-training quantization (PTQ), offers a way to lower computational costs. Given the resource-intensive nature of QAT, which often requires retraining and access to the original dataset [1], our focus is on PTQ. While many quantization methods are designed for general DNNs [35], [36], [37], some recent approaches have been tailored specifically for LLMs [14], [38].

These methods can be applied to weights, activation values, or both, and some even combine quantization with pruning to create more efficient accelerators [39]. Mixed-precision quantization is a popular trend, used across various hardware platforms like GPUs [35], [36], [38], [40] and FPGAs [37]. This technique involves using mixed precision for different parts of the network, such as different layers, tensors, or even weights (e.g., outliers), allowing for a more nuanced and efficient approach to quantization.

Outlier-aware mixed-precision quantization has recently garnered attention, with approaches like GOBO [8] and BitScaled-NN [40] leading the way. However, these methods often struggle with aligning ordinary weights with outliers, resulting in increased computational complexity and memory demands. Techniques such as ANT [19] and Olive [9] have successfully addressed this alignment issue, demonstrating full compatibility with LLMs. Some approaches [14] require a calibration dataset to identify weights that significantly impact activations (considering input) and designate these as outliers, enabling channel-wise (or group-of-channel) quantization. Given the value of datasets for many applications and the potential for variations between datasets to skew outlier identification, as well as the more granular and less hardware-friendly nature of channel-wise quantization compared to tensor-wise quantization, these methods may not be optimal. The proposed WOLF-based quantization achieves tensor-wise alignment of ordinary weights with outliers without requiring a calibration dataset. It supports uniform quantization methods, including segment INT4 or INT8, and mixed INT8/INT4 quantization. The latest work, SPARK [41], has achieved lossless compression for 95% of weights with INT8 quantization. This method can be applied on top of our segment INT8 quantization.

B. Fault Tolerance

Fault tolerance is paramount for deploying DNNs on memory systems susceptible to faults, such as DRAM and NVMs. NVMs are particularly prone to manufacturing- and aging-related hard faults because of their nascent fabrication processes and the challenges of technology scaling [30]. In addition, process variation causes non-uniformity, predisposing some cells to hard faults, while soft faults may arise from crosstalk or environmental noise [30]. In the context of AI acceleration, a number of fault-injection and test frameworks have been studied. Ref. [42] is a comprehensive survey of these techniques—covering diverse model architectures, fault models, and validation methodologies—thereby enabling systematic evaluation of reliability solutions.

A variety of countermeasures have been explored for DNN accelerators, including redundant substitution [43], fault-tolerant training [44] or pruning [45], error-correction encodings [10], [31], and checksum methods [5], [6]. Some approaches also prevent faults from producing extreme values by constraining the range of weights or activations [4]. Solutions such as RRAMedy [44] integrate self-test, remapping, and selective retraining to protect RRAM-based accelerators from both permanent and soft faults. However, retraining-centric techniques, though effective for small or medium models, become impractical for large-scale LLMs because of their prohibitive training overhead.

Compared with conventional DNNs, LLMs introduce an additional challenge: heightened sensitivity to outliers. Outliers tend to cluster, which diminishes the effectiveness of fault-tolerance schemes that assume a uniform fault distribution. The proposed WOLF framework addresses this issue with specialized mechanisms tailored to LLMs. It corrects clustered outlier faults while exploiting the inherent resilience of LLMs to ignore errors in non-outlier weights. Moreover, by using shared hashes, WOLF achieves exceptionally low area overhead.

X. CONCLUSION

We present WOLF, an innovative algorithm-architecture co-design that enables the deployment of large language models (LLMs) on non-volatile memories (NVMs) in environments with unreliable and restricted memory. WOLF introduces a dual-purpose quantization and fault tolerance approach, effectively managing outliers and faults in LLMs. Its Bloom Filter-based weight-level map, the WOLF map, allows for distinct quantization of outliers and regular weights while ensuring fault tolerance. WOLF's fault tolerance scheme, the first of its kind, addresses hard faults and leverages the localized clustering of outliers to enhance error correction, significantly increasing the correctable error rate. The architecture facilitates efficient parallel access and low-latency interpretation, extending the capabilities of INT4 quantization to maintain accuracy at a Bit Error Rate (BER) of 1×10^{-2} . Note that our methods are only suitable for high BERs (ranging from 1×10^{-4} to 1×10^{-2}). At low BERs (lower than 1×10^{-5}), the WOLF method incurs some overhead due to the fixed overhead of its submaps. This

research establishes a new standard for deploying LLMs on NVMs in memory-restricted and unreliable hardware.

ACKNOWLEDGMENT

The authors sincerely thank the reviewers for their insightful comments and constructive suggestions. We also thank Dr. Junjun Yin for her valuable suggestions.

REFERENCES

- [1] W. X. Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.
- [2] S. Zhang et al., "OPT: Open pre-trained transformer language models," 2022, *arXiv:2205.01068*.
- [3] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *Proc. ICML*, 2023, pp. 38087–38099.
- [4] L.-H. Hoang, M. A. Hanif, and M. Shafique, "FT-ClipAct: Resilience analysis of deep neural networks and improving their fault tolerance using clipped activation," in *Proc. IEEE Des., Automat. Test Europe Conf. Exhib. (DATE)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 1241–1246.
- [5] K. Zhao et al., "FT-CNN: Algorithm-based fault tolerance for convolutional neural networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1677–1689, Jul. 2021.
- [6] B. Feinberg, S. Wang, and E. Ipek, "Making memristive neural network accelerators reliable," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Piscataway, NJ, USA: IEEE Press, 2018, pp. 52–65.
- [7] D. Kline, J. Zhang, R. Melhem, and A. K. Jones, "Flower and fame: A low overhead bit-level fault-map and fault-tolerance approach for deeply scaled memories," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 356–368.
- [8] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, "GOBO: Quantizing attention-based NLP models for low latency and energy efficient inference," in *Proc. 53rd Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 811–824.
- [9] C. Guo et al., "Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization," in *Proc. ISCA*, 2023, pp. 1–15.
- [10] J. Zhang et al., "WESCO: Weight-encoded reliability and security co-design for in-memory computing systems," in *Proc. IEEE Comput. Soc. Annu. Symp. VLSI (ISVLSI)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 296–301.
- [11] S. Schechter, G. H. Loh, K. Strauss, and D. Burger, "Use ECP, not ECC, for hard failures in resistive memories," *ACM SIGARCH Comput. Archit. News*, vol. 38, no. 3, pp. 141–152, 2010.
- [12] P. J. Nair, D.-H. Kim, and M. K. Qureshi, "Archshield: Architectural framework for assisting dram scaling by tolerating high error rates," *ACM SIGARCH Comput. Archit. News*, vol. 41, no. 3, pp. 72–83, 2013.
- [13] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," 2022, *arXiv:2210.17323*.
- [14] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, "AWQ: Activation-aware weight quantization for LLM compression and acceleration," 2023.
- [15] OpenAI, "Hello gpt-4o," Accessed: Apr. 20, 2024. [Online]. Available: <https://openai.com/index/hello-gpt-4o/>
- [16] J. Devlin, M.-W. Chang, K. Lee, and K. T. Bert, "Pre-training of deep bidirectional transformers for language understanding," in *Proc. 2019 Conf. North Amer. Chap. Assoc. Comput. Linguistics: Hum. Lang. Technol.*, vol. 1, 2019, pp. 4171–4186.
- [17] Meta AI, "Llama 3," Accessed: Apr. 30, 2024. [Online]. Available: <https://llama.meta.com/llama3/>
- [18] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, 1970.
- [19] C. Guo et al., "ANT: Exploiting adaptive numerical data type for low-bit deep neural network quantization," in *Proc. Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 1414–1433.
- [20] B. Reagen et al., "Minerva: Enabling low-power, highly-accurate deep neural network accelerators," *ACM SIGARCH Comput. Archit. News*, vol. 44, no. 3, pp. 267–278, 2016.
- [21] S. Kannan, J. Rajendran, R. Karri, and O. Sinanoglu, "Sneak-path testing of crossbar-based nonvolatile random access memories," *IEEE Trans. Nanotechnol.*, vol. 12, no. 3, pp. 413–426, May 2013.
- [22] T. Nirschl et al., "Write strategies for 2 and 4-bit multi-level phase-change memory," in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, Piscataway, NJ, USA: IEEE Press, 2007, pp. 461–464.
- [23] A. Paszke et al., "Pytorch: An imperative style, high-performance deep learning library," in *Proc. NIPS*, 2019, p. 32.
- [24] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [25] T. Le Scao et al., "Bloom: A 176b-parameter open-access multilingual language model," 2022, *arXiv:2211.05100*.
- [26] H. Touvron et al., "LLAMA 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
- [27] Y. Bisk, R. Zellers, R. Le Bras, J. Gao, and Y. Choi, "PIQA: Reasoning about physical commonsense in natural language," in *Proc. AAAI*, 2020.
- [28] P. Clark et al., "Think you have solved question answering? Try Arc, the AI2 reasoning challenge," 2018, *arXiv:1803.05457*.
- [29] ModelTC, "Repositories," Accessed: Jun. 19, 2022. [Online]. Available: <https://huggingface.co/ModelTC>
- [30] J. Zhang et al., "Realizing extreme endurance through fault-aware wear leveling and improved tolerance," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 964–976.
- [31] R. W. Hamming, "Error detecting and error correcting codes," *Bell Syst. Tech. J.*, vol. 29, no. 2, pp. 147–160, 1950.
- [32] X. Dong, C. Xu, Y. Xie, and N. P. Jouppi, "NVSim: A circuit-level performance, energy, and area model for emerging nonvolatile memory," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 31, no. 7, pp. 994–1007, Jul. 2012.
- [33] Y. Xie and J. Zhao, "Emerging memory technologies," *IEEE Micro*, vol. 39, no. 1, pp. 6–7, Jan./Feb. 2019.
- [34] A. Gholami et al., "A survey of quantization methods for efficient neural network inference," 2021.
- [35] S. Huang et al., "Mixed precision quantization for RERAM-based DNN inference accelerators," in *Proc. ASP-DAC*, 2021, pp. 372–377.
- [36] R. Zhao, Y. Hu, J. Dotzel, C. De Sa, and Z. Zhang, "Improving neural network quantization without retraining using outlier channel splitting," in *Proc. ICML*, PMLR, 2019, pp. 7543–7552.
- [37] S. Zeng et al., "A fine-grained sparse accelerator for multi-precision DNN," in *Proc. FPGA*, 2019, pp. 185–185.
- [38] J. Xu, S. Hu, J. Yu, X. Liu, and H. Meng, "Mixed precision quantization of transformer language models for speech recognition," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 7383–7387.
- [39] P. Hu, X. Peng, H. Zhu, M. M. S. Aly, and J. Lin, "OPQ: Compressing deep neural networks with one-shot pruning-quantization," in *Proc. AAAI*, vol. 35, 2021, pp. 7780–7788.
- [40] S. Jain, S. Venkataramani, V. Srinivasan, J. Choi, K. Gopalakrishnan, and L. Chang, "BiScaled-DNN: Quantizing long-tailed datastructures with two scale factors for deep neural networks," in *Proc. DAC*, 2019, pp. 1–6.
- [41] F. Liu et al., "Spark: Scalable and precision-aware acceleration of neural networks via efficient encoding," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Piscataway, NJ, USA: IEEE Press, 2024, pp. 1029–1042.
- [42] F. Su, C. Liu, and H.-G. Stratigopoulos, "Testability and dependability of AI hardware: Survey, trends, challenges, and perspectives," *IEEE Des. Test; IEEE Des. Test Comput.*, vol. 40, no. 2, pp. 8–58, Apr. 2023.
- [43] L. Xia et al., "Stuck-at fault tolerance in RRAM computing systems," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 8, no. 1, pp. 102–115, Mar. 2018.
- [44] W. Li, Y. Wang, H. Li, and X. Li, "RRAMedy: Protecting ReRAM-based neural network from permanent and soft faults during its lifetime," in *Proc. IEEE 37th Int. Conf. Comput. Des. (ICCD)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 91–99.
- [45] C.-Y. Chen and K. Chakrabarty, "Pruning of deep neural networks for fault-tolerant memristor-based accelerators," in *Proc. 58th ACM/IEEE Des. Automat. Conf. (DAC)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 889–894.



Chong Wang received the B.S. degree in electronic engineering from Tsinghua University, in 2022. He is currently working toward the Ph.D. degree with the EE Department, Tsinghua University. His research mainly focuses on efficient deep learning algorithms and architecture.



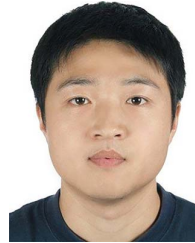
Wanyi Fu received the B.S. degree from the University of Minnesota, in 2013, and the M.S. and Ph.D. degrees in 2016 and 2021, respectively, from Duke University, all in electronic engineering. Currently, she is an Assistant Researcher with Tsinghua University. Her research mainly focuses on efficient deep learning algorithms and simulation.



Jiangwei Zhang (Member, IEEE) received the B.S., M.S., and Ph.D. degrees from the National University of Defense Technology, in 2012, 2014, and 2018, respectively. From 2016 to 2018, he was a Visiting Ph.D. Student with the Department of Electrical and Computer Engineering, University of Pittsburgh, PA, USA. From 2021 to 2023, he was an Assistant Researcher with the Department of Electronic Engineering, Tsinghua University. Currently, he is an Associate Professor with the State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Science. His research mainly focuses on efficient deep learning algorithms and architecture, memory reliability, and privacy-preserving inference.



Shiyao Li received the B.S. degree in electronic engineering from Tsinghua University, in 2020. He is currently working toward the Ph.D. degree with the EE Department, Tsinghua University. His research mainly focuses on continual learning and large language models.



research interests include computer architecture, processor security, data center server architecture, and AI security.



research interests include radar polarimetry, feature extraction, target detection, and target classification.



Yu Wang (Fellow, IEEE) received the B.S. and Ph.D.(Hons.) degrees from Tsinghua University, Beijing, in 2002 and 2007, respectively. Currently, he is a Tenured Professor with the Department of Electronic Engineering, Tsinghua University. His research interests include application specific accelerators, brain inspired computing, parallel circuit analysis, and power/reliability aware system design. He has authored and coauthored more than 300 papers in refereed journals and conferences. He has received Best Paper Award in ASPDAC 2019, FPGA 2017, NVMSA 2017, ISVLSI 2012, and Best Poster Award in HEART 2012 with 10 Best Paper Nominations. He is a recipient of DAC under 40 innovator award (2018) and IBM X10 Faculty Award (2010). He served as the TPC Chair, a Finance Chair, a Track Chair, a PC Member, and editors for top EDA and FPGA conferences and journals. He is the Co-Founder of Deepphi Tech (acquired by Xilinx in 2018), which is a leading deep learning computing platform provider.

Rui Hou received the B.S. and M.S. degrees from Harbin Institute of Technology, Harbin, China, in 2001 and 2003, respectively, and the Ph.D. degree from the Institute of Computing Technology (ICT), Chinese Academy of Sciences (CAS), Beijing, China, in 2007, all in computer architecture. He is a Professor and the Director of the State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, CAS. He published over 40 papers in international conferences and journals, and got more than 50 patents. His

Jian Yang (Senior Member, IEEE) received the B.S. and M.S. degrees from the Northwestern Polytechnical University, Xi'an, China, in 1985 and 1990, respectively, and the Ph.D. degree from Niigata University, Niigata, Japan, in 1999. In 1985, he joined the Department of Applied Mathematics, Northwestern Polytechnical University. From 1999 to 2000, he was an Assistant Professor with Niigata University. In 2000, he joined the Department of Electronic Engineering, Tsinghua University, Beijing, China, where he was promoted as a Full

Professor, in 2002. He has authored or coauthored more than 300 papers and received many awards. His research interests include radar polarimetry, feature extraction, target detection, and target classification.